

```
import pandas as pd

# Load the comments.csv file into a DataFrame
df_comments = pd.read_csv('/content/comments.csv')
```

```
# Display the first 5 rows of the DataFrame
display(df_comments.head())
```

	id	comment	User id	Photo id	created Timestamp	posted date	emoji used	Hashtags used count	
0	1	unde at dolore	2	1	13-04-2023 08:04	April 14	yes	1	
1	2	quae ea ducimus	3	1	13-04-2023 08:04	April 14	no	2	
2	3	alias a voluptatum	5	1	13-04-2023 08:04	April 14	no	4	
3	4	facere suscipit sunt	14	1	13-04-2023 08:04	April 14	yes	2	
4	5	totam eligendi quaerat	17	1	13-04-2023 08:04	April 14	yes	1	

```
# Display information about the DataFrame, including data types and non-null values
df_comments.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7488 entries, 0 to 7487
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     7488 non-null  int64
1   comment                7488 non-null  object
2   User id                7488 non-null  int64
3   Photo id               7488 non-null  int64
4   created Timestamp      7488 non-null  object
5   posted date            7488 non-null  object
6   emoji used             7488 non-null  object
7   Hashtags used count    7488 non-null  int64
dtypes: int64(4), object(4)
memory usage: 468.1+ KB
```

```
# Convert 'created Timestamp' to datetime objects
df_comments['created Timestamp'] = pd.to_datetime(df_comments['created Timestamp'], format='%d-%m-%Y %H:%M')
```

```
# Display the first 5 rows of the DataFrame after conversion
display(df_comments.head())
```

	id	comment	User id	Photo id	created Timestamp	posted date	emoji used	Hashtags used count	hour	day_of_week	month	
0	1	unde at dolore	2	1	2023-04-13 08:04:00	April 14	yes	1	8	Thursday	April	
1	2	quae ea ducimus	3	1	2023-04-13 08:04:00	April 14	no	2	8	Thursday	April	
2	3	alias a voluptatum	5	1	2023-04-13 08:04:00	April 14	no	4	8	Thursday	April	

```
# Display information about the DataFrame to verify data types
df_comments.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7488 entries, 0 to 7487
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                     7488 non-null  int64
1   comment                7488 non-null  object
2   User id                7488 non-null  int64
3   Photo id               7488 non-null  int64
4   created Timestamp      7488 non-null  datetime64[ns]
5   posted date            7488 non-null  object
6   emoji used             7488 non-null  object
7   Hashtags used count    7488 non-null  int64
8   hour                   7488 non-null  int32
```

```
9   day_of_week      7488 non-null  object
10  month            7488 non-null  object
dtypes: datetime64[ns](1), int32(1), int64(4), object(5)
memory usage: 614.4+ KB
```

```
# Extract hour, day of the week, and month from 'created Timestamp'
df_comments['hour'] = df_comments['created Timestamp'].dt.hour
df_comments['day_of_week'] = df_comments['created Timestamp'].dt.day_name()
df_comments['month'] = df_comments['created Timestamp'].dt.month_name()
```

```
# Check for missing values
display(df_comments.isnull().sum())
```

	0
id	0
comment	0
User id	0
Photo id	0
created Timestamp	0
posted date	0
emoji used	0
Hashtags used count	0
hour	0
day_of_week	0
month	0

dtype: int64

```
# Display the first few rows with the new temporal features
display(df_comments.head())
```

	id	comment	User id	Photo id	created Timestamp	posted date	emoji used	Hashtags used count	hour	day_of_week	month
0	1	unde at dolore	2	1	2023-04-13 08:04:00	April 14	yes	1	8	Thursday	April
1	2	quae ea ducimus	3	1	2023-04-13 08:04:00	April 14	no	2	8	Thursday	April
2	3	alias a voluptatum	5	1	2023-04-13 08:04:00	April 14	no	4	8	Thursday	April

```
# Display information about the DataFrame to verify the new columns
df_comments.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7488 entries, 0 to 7487
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    7488 non-null  int64
1   comment              7488 non-null  object
2   User id              7488 non-null  int64
3   Photo id            7488 non-null  int64
4   created Timestamp    7488 non-null  datetime64[ns]
5   posted date          7488 non-null  object
6   emoji used           7488 non-null  object
7   Hashtags used count  7488 non-null  int64
8   hour                 7488 non-null  int32
9   day_of_week          7488 non-null  object
10  month                7488 non-null  object
dtypes: datetime64[ns](1), int32(1), int64(4), object(5)
memory usage: 614.4+ KB
```

```
# Convert 'created Timestamp' to datetime objects
df_comments['created Timestamp'] = pd.to_datetime(df_comments['created Timestamp'], format='%d-%m-%Y %H:%M')
```

```
# Removed: Conversion of 'posted date' was causing an OutOfBoundsDatetime error as it lacks year information.
# df_comments['posted date'] = pd.to_datetime(df_comments['posted date'], infer_datetime_format=True)
```

```
# Display the first 5 rows of the DataFrame after conversion
display(df_comments.head())
```

	id	comment	User id	Photo id	created Timestamp	posted date	emoji used	Hashtags used count	hour	day_of_week	month	
0	1	unde at dolore	2	1	2023-04-13 08:04:00	April 14	yes	1	8	Thursday	April	
1	2	quae ea ducimus	3	1	2023-04-13 08:04:00	April 14	no	2	8	Thursday	April	
2	3	alias a voluptatum	5	1	2023-04-13 08:04:00	April 14	no	4	8	Thursday	April	

```
# Display information about the DataFrame to verify data types
df_comments.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7488 entries, 0 to 7487
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   id                    7488 non-null  int64
1   comment              7488 non-null  object
2   User id              7488 non-null  int64
3   Photo id            7488 non-null  int64
4   created Timestamp    7488 non-null  datetime64[ns]
5   posted date          7488 non-null  object
6   emoji used           7488 non-null  object
7   Hashtags used count  7488 non-null  int64
8   hour                 7488 non-null  int32
9   day_of_week          7488 non-null  object
10  month                7488 non-null  object
dtypes: datetime64[ns](1), int32(1), int64(4), object(5)
memory usage: 614.4+ KB
```

```
# Extract hour, day of the week, and month from 'created Timestamp'
df_comments['hour'] = df_comments['created Timestamp'].dt.hour
df_comments['day_of_week'] = df_comments['created Timestamp'].dt.day_name()
df_comments['month'] = df_comments['created Timestamp'].dt.month_name()
```

```
# Check for missing values
display(df_comments.isnull().sum())
```

	0
id	0
comment	0
User id	0
Photo id	0
created Timestamp	0
posted date	0
emoji used	0
Hashtags used count	0
hour	0
day_of_week	0
month	0

dtype: int64

```
# Display the first few rows with the new temporal features
display(df_comments.head())
```

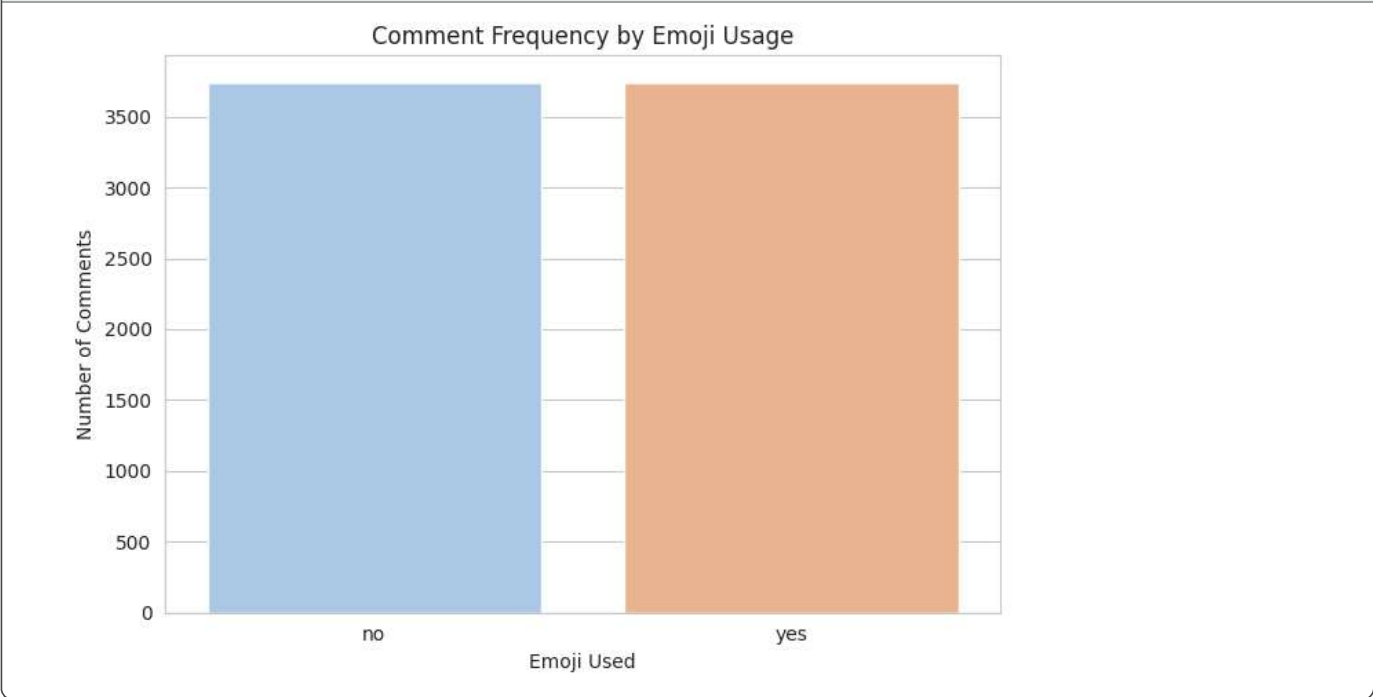
	id	comment	User id	Photo id	created Timestamp	posted date	emoji used	Hashtags used count	hour	day_of_week	month
0	1	unde at dolorem	2	1	2023-04-13 08:04:00	April 14	yes	1	8	Thursday	April
1	2	quae ea ducimus	3	1	2023-04-13 08:04:00	April 14	no	2	8	Thursday	April
2	3	alias a voluptatum	5	1	2023-04-13 08:04:00	April 14	no	4	8	Thursday	April

```
# Display information about the DataFrame to verify the new columns
df_comments.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7488 entries, 0 to 7487
Data columns (total 11 columns):
#   Column              Non-Null Count  Dtype
---  ---
0    id                  7488 non-null   int64
1   comment             7488 non-null   object
2   User id              7488 non-null   int64
3   Photo id             7488 non-null   int64
4   created Timestamp    7488 non-null   datetime64[ns]
5   posted date          7488 non-null   object
6   emoji used           7488 non-null   object
7   Hashtags used count  7488 non-null   int64
8   hour                 7488 non-null   int32
9   day_of_week          7488 non-null   object
10  month                7488 non-null   object
dtypes: datetime64[ns](1), int32(1), int64(4), object(5)
memory usage: 614.4+ KB
```

```
# Analyze comment frequency based on 'emoji used'
emoji_engagement = df_comments.groupby('emoji used')['comment'].count().reset_index()
emoji_engagement.columns = ['Emoji Used', 'Number of Comments']

plt.figure(figsize=(7, 5))
sns.barplot(x='Emoji Used', y='Number of Comments', data=emoji_engagement, hue='Emoji Used', palette='pastel', legend=False)
plt.title('Comment Frequency by Emoji Usage')
plt.xlabel('Emoji Used')
plt.ylabel('Number of Comments')
plt.tight_layout()
plt.show()
```



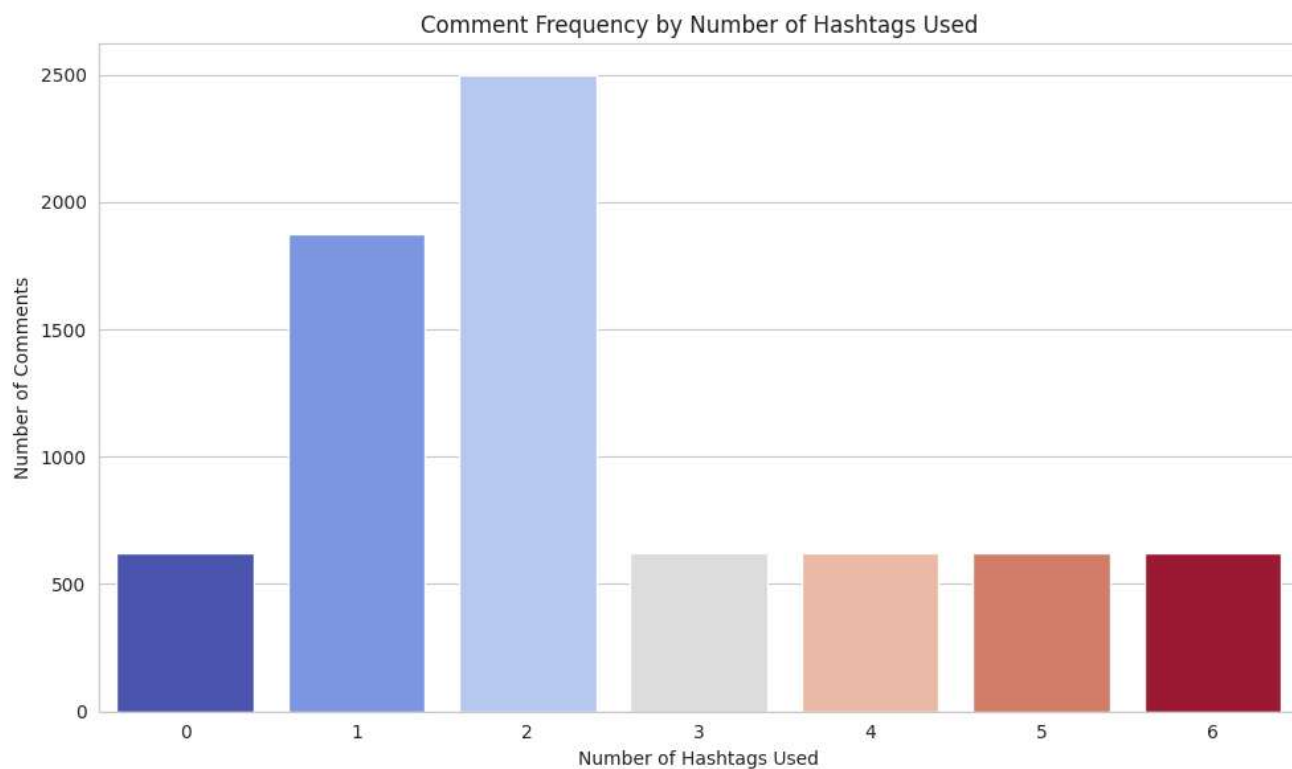
```
# Analyze comment frequency based on 'Hashtags used count'
hashtag_engagement = df_comments.groupby('Hashtags used count')['comment'].count().reset_index()
```

```

hashtag_engagement.columns = ['Hashtags Used Count', 'Number of Comments']

plt.figure(figsize=(10, 6))
sns.barplot(x='Hashtags Used Count', y='Number of Comments', data=hashtag_engagement, hue='Hashtags Used Count', palette='c
plt.title('Comment Frequency by Number of Hashtags Used')
plt.xlabel('Number of Hashtags Used')
plt.ylabel('Number of Comments')
plt.tight_layout()
plt.show()

```



```

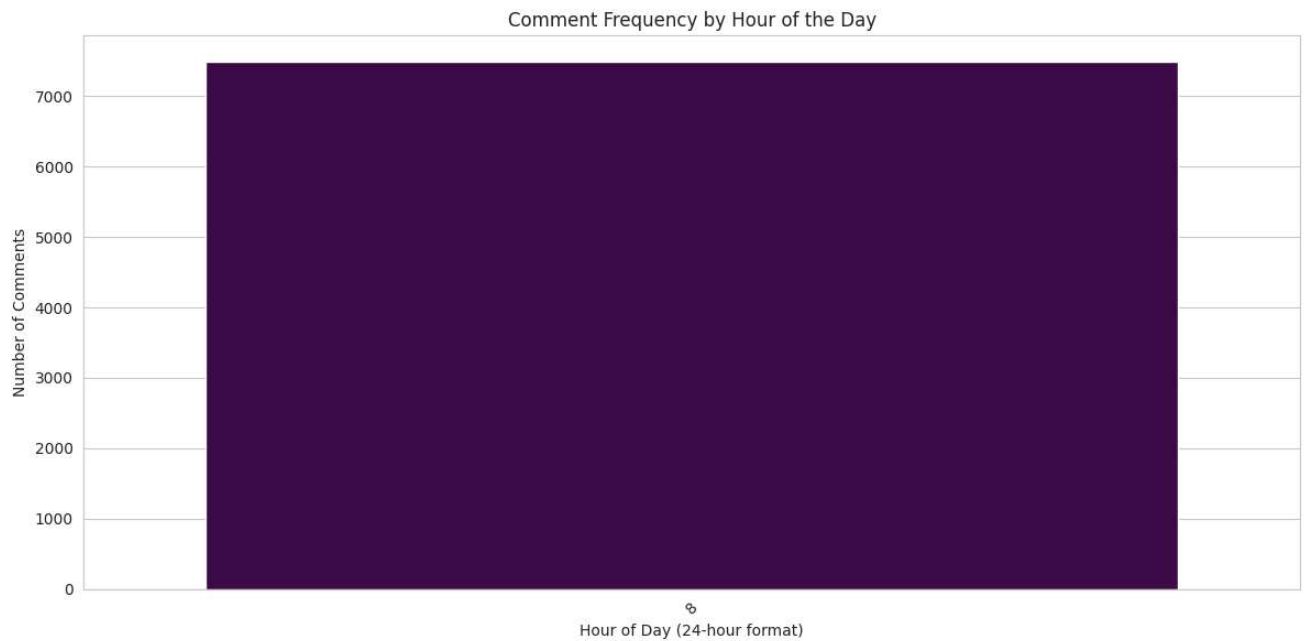
import matplotlib.pyplot as plt
import seaborn as sns

# Set the style for the plots
sns.set_style('whitegrid')

# Analyze comment frequency by hour of the day
comments_by_hour = df_comments['hour'].value_counts().sort_index()

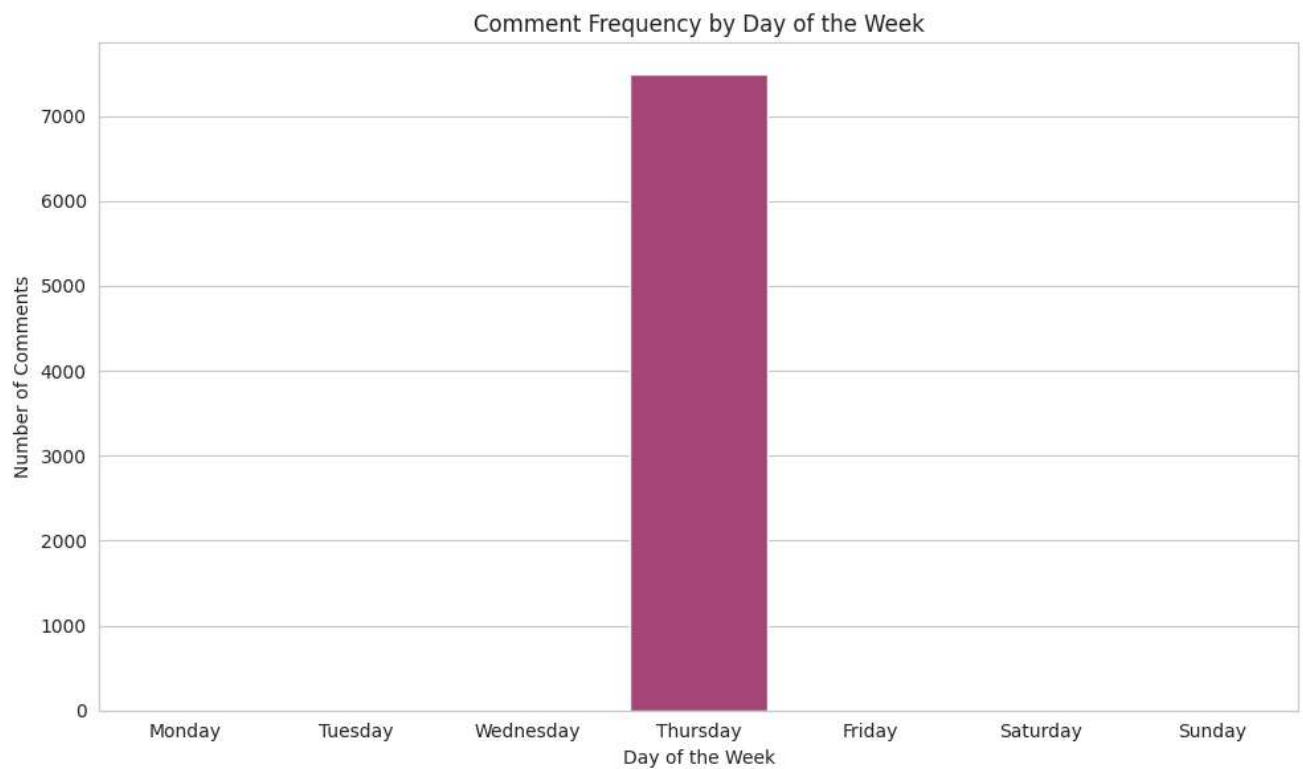
plt.figure(figsize=(12, 6))
sns.barplot(x=comments_by_hour.index, y=comments_by_hour.values, hue=comments_by_hour.index, palette='viridis', legend=False)
plt.title('Comment Frequency by Hour of the Day')
plt.xlabel('Hour of Day (24-hour format)')
plt.ylabel('Number of Comments')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



```
# Analyze comment frequency by day of the week
# Ensure days are in the correct order
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
comments_by_day = df_comments['day_of_week'].value_counts().reindex(day_order)

plt.figure(figsize=(10, 6))
sns.barplot(x=comments_by_day.index, y=comments_by_day.values, hue=comments_by_day.index, palette='magma', legend=False)
plt.title('Comment Frequency by Day of the Week')
plt.xlabel('Day of the Week')
plt.ylabel('Number of Comments')
plt.tight_layout()
plt.show()
```



```
import matplotlib.pyplot as plt
import seaborn as sns

# Set the style for the plots
sns.set_style('whitegrid')

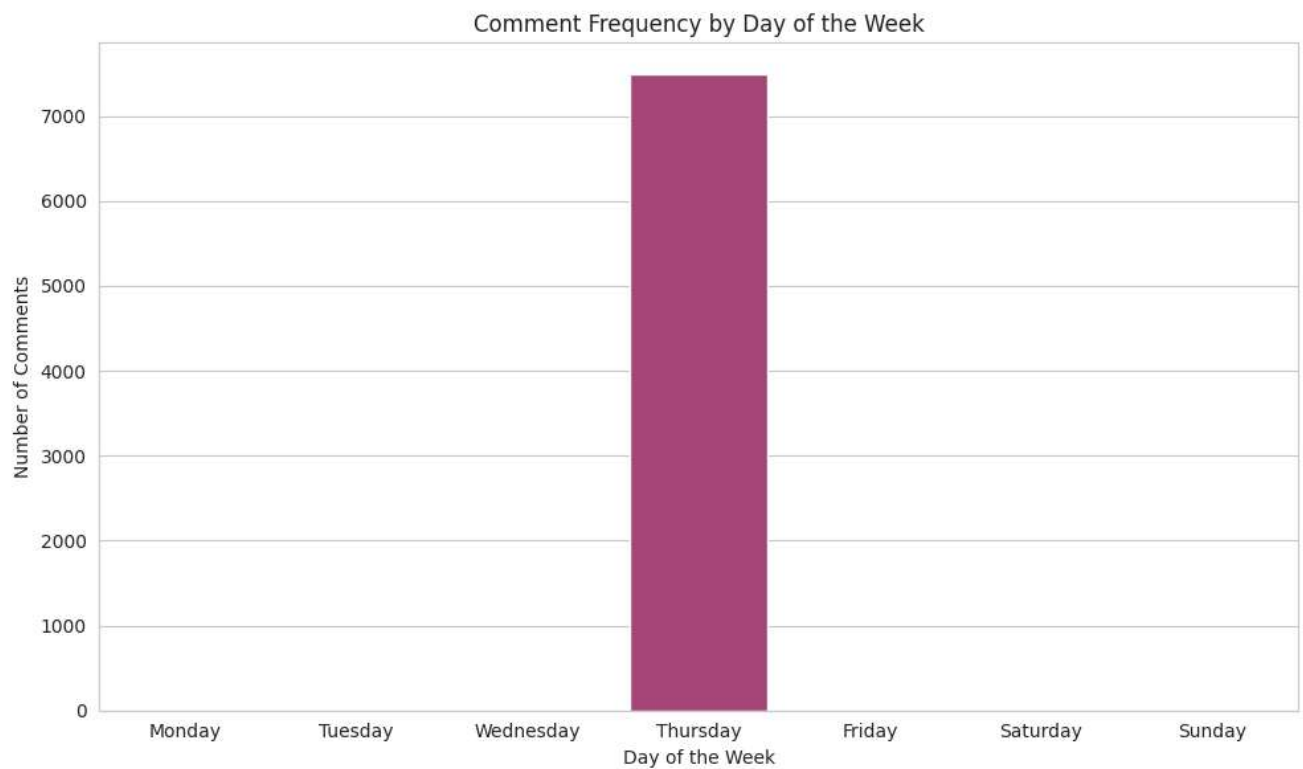
# Analyze comment frequency by hour of the day
comments_by_hour = df_comments['hour'].value_counts().sort_index()

plt.figure(figsize=(12, 6))
sns.barplot(x=comments_by_hour.index, y=comments_by_hour.values, hue=comments_by_hour.index, palette='viridis', legend=False)
plt.title('Comment Frequency by Hour of the Day')
plt.xlabel('Hour of Day (24-hour format)')
plt.ylabel('Number of Comments')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
# Analyze comment frequency by day of the week
# Ensure days are in the correct order
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
comments_by_day = df_comments['day_of_week'].value_counts().reindex(day_order)

plt.figure(figsize=(10, 6))
sns.barplot(x=comments_by_day.index, y=comments_by_day.values, hue=comments_by_day.index, palette='magma', legend=False)
plt.title('Comment Frequency by Day of the Week')
plt.xlabel('Day of the Week')
plt.ylabel('Number of Comments')
plt.tight_layout()
plt.show()
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set the style for the plots
sns.set_style('whitegrid')

# Analyze comment frequency by hour of the day
comments_by_hour = df_comments['hour'].value_counts().sort_index()

plt.figure(figsize=(12, 6))
sns.barplot(x=comments_by_hour.index, y=comments_by_hour.values, hue=comments_by_hour.index, palette='viridis', legend=False)
plt.title('Comment Frequency by Hour of the Day')
plt.xlabel('Hour of Day (24-hour format)')
plt.ylabel('Number of Comments')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
# Analyze comment frequency by day of the week
# Ensure days are in the correct order
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
comments_by_day = df_comments['day_of_week'].value_counts().reindex(day_order)

plt.figure(figsize=(10, 6))
sns.barplot(x=comments_by_day.index, y=comments_by_day.values, hue=comments_by_day.index, palette='magma', legend=False)
plt.title('Comment Frequency by Day of the Week')
plt.xlabel('Day of the Week')
plt.ylabel('Number of Comments')
plt.tight_layout()
plt.show()
```

Comment Frequency by Day of the Week

```
df_comments['created Timestamp'] = pd.to_datetime(df_comments['created Timestamp'], format='%d-%m-%Y %H:%M')

# The conversion of 'posted date' was causing an OutOfBoundsDatetime error due to missing year information.
# df_comments['posted date'] = pd.to_datetime(df_comments['posted date'], infer_datetime_format=True)
```

```
# Display the first 5 rows of the DataFrame after conversion
display(df_comments.head())
```

	id	comment	User id	Photo id	created Timestamp	posted date	emoji used	Hashtags used	count
0	1	unde at dolore	2	1	2023-04-13 08:04:00	April 14	yes		1
1	2	quae ea ducimus	3	1	2023-04-13 08:04:00	April 14	no		2
2	3	alias a voluptatum	5	1	2023-04-13 08:04:00	April 14	no		4
3	4	facere suscipit sunt	14	1	2023-04-13 08:04:00	April 14	yes		2
4	5	totam eligendi quaerat	17	1	2023-04-13 08:04:00	April 14	yes		1

```
# Display information about the DataFrame to verify data types
df_comments.info()
```

<class 'pandas.core.frame.DataFrame'>									
RangeIndex: 7488 entries, 0 to 7487									
Data columns (non-null) columns				Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
#	Column	Non-Null	Count	Dtype	Day of the Week				
0	id	7488	non-null	int64					
1	comment	7488	non-null	object					
2	User id	7488	non-null	int64					
3	Photo id	7488	non-null	int64					
4	created Timestamp	7488	non-null	datetime64[ns]					
5	posted date	7488	non-null	object					